

Python Best Practices Guide

1. Code Style and PEP 8

PEP 8 rules: 4 spaces indentation, max 79 char lines, snake_case for variables/functions, PascalCase for classes, blank lines (2 before top-level, 1 inside classes). Use Black formatter and Ruff linter for automated enforcement.

2. Type Hints and Type Safety

Python type hints (PEP 484) enable static type checking with mypy or pyright. Use typing module: Optional, List, Dict, Union. Use dataclasses or Pydantic for structured data validation. Type hints improve readability and IDE autocompletion.

Python Best Practices Guide

3. Error Handling

Use specific exceptions (ValueError, TypeError), never bare except. Never silently swallow exceptions with pass. Use contextlib.contextmanager for resource cleanup. Log exceptions with traceback. Raise with descriptive messages.

4. Testing with pytest

pytest best practices: tests/ directory mirroring source, descriptive test names, fixtures in conftest.py, parametrize for data-driven tests, aim for >80% coverage. Mock externals with unittest.mock. Run: `pytest tests/ -v --cov=src --cov-report=term-missing`

Python Best Practices Guide

5. Requirements for Production Python Systems

Requirements: (1) Dependency Isolation - virtual environments or containers, (2) Reproducible Builds - pinned dependencies in requirements.txt, (3) Configuration Management - environment variables via python-dotenv, (4) Logging - structured logging with levels DEBUG/INFO/WARNING/ERROR, (5) Testing - comprehensive test suite with CI/CD, (6) Documentation - docstrings, README, API reference, (7) Error Handling - no silent failures, proper exception propagation, (8) Security - no hardcoded secrets, regular vulnerability scanning.